

End-to-End ML Toy Project

Module 13 Study Notes • Practical Implementation

1. The Plan of Attack

To successfully build a machine learning model from scratch and deploy it, one must follow a structured plan of attack based on the Machine Learning Development Life Cycle (MLDLC). A toy project helps bridge the gap between theoretical knowledge and practical execution. The workflow usually consists of:

1. Data Preprocessing & Exploratory Data Analysis (EDA)
2. Feature Selection and Extraction
3. Train-Test Split
4. Model Training & Evaluation
5. Model Exporting & Cloud Deployment

2. Preprocessing & EDA

Before an algorithm can learn from data, the data must be thoroughly examined and cleaned.

- **Preprocessing:** This involves addressing missing values, handling categorical data (converting text to numbers), and standardizing or scaling numerical features.
- **Exploratory Data Analysis (EDA):** A critical phase where visual tools (like scatterplots and histograms) are utilized to understand data distribution, identify extreme outliers, and uncover hidden relationships between variables.

Scatterplot Example: A scatterplot maps two numerical variables against each other. It is extremely useful for visually determining if classes are easily separable (e.g., distinguishing between approved and rejected loan applications based on income and credit score).

3. Feature Selection & Extracting Columns

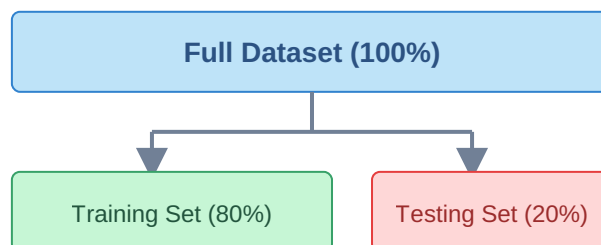
Not all data collected is useful for making predictions. Feature selection is the process of intentionally keeping only the most relevant inputs to reduce noise and computation time.

- **Inputs (Features / Independent Variables):** Typically extracted into a matrix usually denoted as capital **X**.
- **Output (Target / Dependent Variable):** Extracted into a vector usually denoted as lowercase **y**.

4. Train-Test Split

Never train and evaluate a model on the exact same data. If you do, the model might just memorize the answers (Overfitting) rather than learning the underlying concepts.

- **Training Set:** Typically 70% to 80% of the data. This is given to the algorithm to learn the relationships and build the mathematical rules.
- **Testing Set:** The remaining 20% to 30%. This is kept hidden from the model during training. It acts as an "exam" to evaluate how well the model predicts entirely unseen data.



5. Evaluating the Model

Once the model is trained, it is scored based on its performance on the Testing Set. For classification tasks, the most straightforward metric is the **Accuracy Score**.

- **Accuracy Score:** The ratio of correctly predicted observations to the total number of observations. For example, if the model correctly predicts 90 out of 100 test cases, it has an accuracy score of 0.90 (or 90%).

6. Exporting and Deploying the Model

Training a model in a local Jupyter Notebook is only half the battle. To make the model accessible to end-users (like a web app), it must be exported and deployed to the cloud.

- **Exporting:** The trained model object is serialized and saved as a file on the local machine (frequently as a `.pkl` / Pickle file in Python). This file contains all the learned mathematical weights and rules.
- **Cloud Deployment:** The exported model file is uploaded to a cloud hosting platform (such as Heroku, AWS, or GCP) alongside a lightweight web framework (like Flask or Streamlit). The cloud server listens for incoming data from users, runs it through the model file, and returns the prediction instantly.